

République Algérienne Démocratique Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene
USTHB



Faculté d'informatique

M1 – IL

2025/2026

**Mini-projet multimédia : Pipeline simplifié d'un
encodeur vidéo MPEG-4**

Présenté par :

Nom & Prénom	Matricule
Ladjici Malak Lamisse	222231472401
Aridj Amira	222231468512

Groupe : 3

1. Introduction

Aujourd'hui, la vidéo est partout sur nos téléphones, nos plateformes de streaming, nos réseaux sociaux. Mais derrière chaque vidéo que l'on regarde se cache un mécanisme invisible et pourtant essentiel : la compression vidéo. Sans elle, envoyer une simple vidéo par message serait impossible, et Netflix consommerait une quantité astronomique de bande passante.

Dans le cadre de ce projet, nous avons implémenté en Python un pipeline d'encodage vidéo simplifié, inspiré du standard MPEG-4. L'idée est simple : prendre une séquence d'images brutes, les compresser intelligemment en exploitant les redondances visuelles, et produire un fichier binaire compact qui peut être décodé pour retrouver les images originales.

2. Description du Pipeline

➤ Partie 1 : Pré-traitement

La première étape consiste à convertir chaque frame de l'espace colorimétrique BGR vers YCbCr. Cet espace sépare la luminance (Y) des informations de chrominance (Cb et Cr), ce qui est essentiel pour la compression car l'œil humain est bien plus sensible aux variations de luminosité qu'aux variations de couleur.

Une fois la conversion effectuée, nous appliquons un sous-échantillonnage chromatique 4:2:0 : les canaux Cb et Cr sont réduits d'un facteur 2 dans chaque dimension (de 128×128 à 64×64), divisant ainsi leur taille par 4. Le canal Y est conservé en pleine résolution. Lors de la reconstruction, Cb et Cr sont redimensionnés par interpolation linéaire avant de recombinaison les canaux.

➤ Partie 2 : Codage Intra-frame (I-frames)

Pour compresser chaque image individuellement, on utilise la DCT (Transformée en Cosinus Discrète), qui est la même méthode utilisée par le format JPEG.

Chaque canal de l'image est découpé en blocs de 8×8 pixels, puis une transformation mathématique est appliquée afin de convertir les valeurs des pixels en composantes fréquentielles.

Le facteur QF contrôle le niveau de compression : plus sa valeur est élevée, plus la compression est importante, ce qui réduit la taille du fichier mais entraîne également une perte de qualité de l'image.

Lors du décodage, il suffit d'appliquer la transformation inverse pour reconstruire l'image.

➤ **Partie 3 :Codage Inter-frame (P-frames)**

Les I-frames compressent chaque image de manière indépendante. Cependant, entre deux images successives d'une vidéo, les variations sont souvent faibles. Cette similarité temporelle est exploitée grâce aux P-frames.

Le canal Y est divisé en macroblocs de 16×16 pixels. Pour chaque macrobloc, on recherche la meilleure correspondance dans une zone voisine de ± 8 pixels à l'aide de la méthode SAD .

Au lieu de stocker l'image complète, on conserve uniquement le vecteur de mouvement ainsi que le résidu, qui est lui aussi compressé à l'aide de la DCT.

Lors du décodage, chaque bloc est reconstruit en combinant la prédiction obtenue par le mouvement avec le résidu après décompression.

➤ **Partie 4 :Codage Entropique**

Une fois toutes les frames encodées, il faut tout regrouper dans un seul fichier binaire compact. Chaque frame I-frame ou P-frame est sérialisée avec pickle puis compressée individuellement avec zlib . Le fichier final commence par un header indiquant le nombre de frames, suivi de chaque frame compressée précédée de sa taille.

Le décodeur fait l'inverse : il lit le header, décompresse chaque frame avec zlib, la déséréalise, puis la reconstruit selon son type (I ou P).

➤ **Part 5 : Évaluation et Visualisation**

La dernière partie consiste à évaluer et visualiser les résultats obtenus. Pour mesurer la qualité de compression, on utilise le PSNR (Peak Signal-to-Noise Ratio) : plus cette valeur est élevée, meilleure est la qualité de l'image reconstruite.

Deux figures sont générées :

- La première illustre les différentes étapes du pipeline de compression : les frames originales, les composantes Y/Cb/Cr, le traitement d'un bloc 8×8 par la DCT, les vecteurs de mouvement, les résidus ainsi que la frame reconstruite.
- La seconde présente les courbes d'analyse, notamment le taux de compression en fonction du facteur QF, ainsi que l'influence de la taille du GOP sur le niveau de compression.

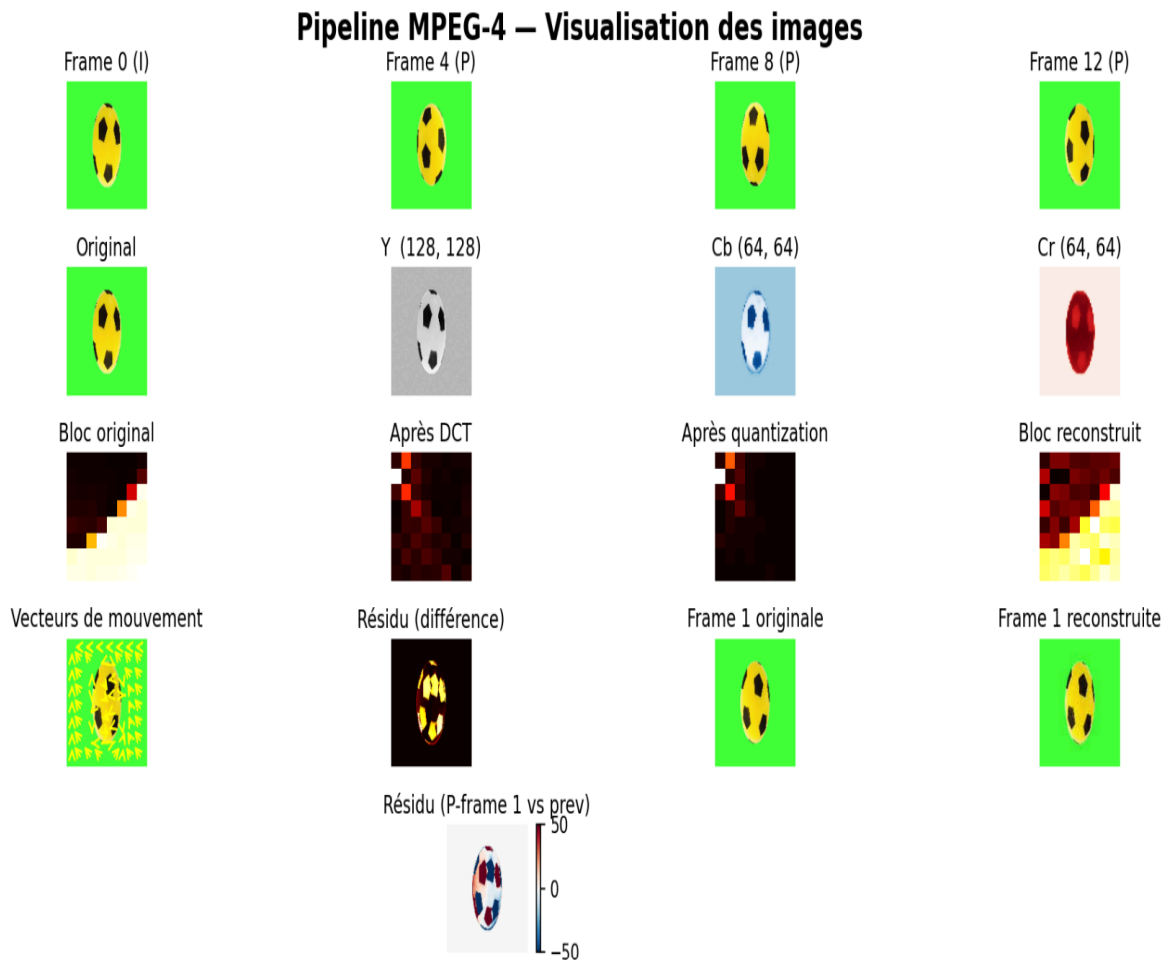


Figure 1 : Visualisation complète du pipeline MPEG-4

3. Choix du Design et Justification

➤ Espace colorimétrique YCbCr et sous-échantillonnage 4:2:0

Le format YCbCr est utilisé car l'œil humain perçoit surtout la luminance (Y) et moins les couleurs (Cb, Cr). Le sous-échantillonnage 4:2:0 réduit donc les informations de couleur d'environ 75 % sans perte visible de qualité. Ce principe est aussi utilisé par JPEG et MPEG.

➤ Matrice de quantification dérivée de JPEG

La matrice de quantification utilisée vient du standard JPEG et peut être ajustée avec le facteur `QUANTIZATION_FACTOR`. Elle conserve surtout les basses fréquences, plus importantes visuellement, et réduit les hautes fréquences, moins perceptibles. Avec la valeur par défaut $QF=10$, on obtient un bon équilibre entre qualité et compression (environ 19x).

➤ Taille de bloc 8x8 pour la DCT

Le bloc 8×8 est le format standard utilisé pour la DCT en compression d'image et vidéo. Il assure un bon compromis entre qualité et efficacité de compression. Des blocs plus grands comprimeraient mieux les données, mais rendraient les artefacts sur les bords plus visibles.

➤ Macroblocs 16x16 pour l'estimation de mouvement

Les macroblocs 16×16 sont le standard utilisé dans MPEG pour la compensation de mouvement. Ils sont assez grands pour représenter un mouvement cohérent, tout en restant adaptés aux mouvements locaux. La fenêtre de recherche de ± 16 pixels convient bien à la vitesse de déplacement du ballon dans la séquence test.

➤ Quantification plus agressive pour les résidus P-frame

Dans `encode_video`, les résidus des P-frames sont quantifiés avec $qf*2$, soit un facteur deux fois plus élevé que celui des I-frames. Cela est possible car les résidus ont une amplitude plus faible que les coefficients bruts des I-frames, donc une quantification plus forte n'entraîne pas de perte visuelle importante.

➤ Serialisation pickle + zlib

Le choix de pickle permet une sérialisation simple et flexible des tableaux NumPy, vecteurs de mouvement et métadonnées sans code complexe. La compression avec zlib (niveau 6) offre un bon compromis entre vitesse et taux de compression. Même si cette méthode n'utilise pas un vrai codage entropique comme Huffman, elle reste pratique pour un projet éducatif et donne de bons résultats de compression.

4. Analyse Expérimentale

Pipeline MPEG-4 – Analyse quantitative

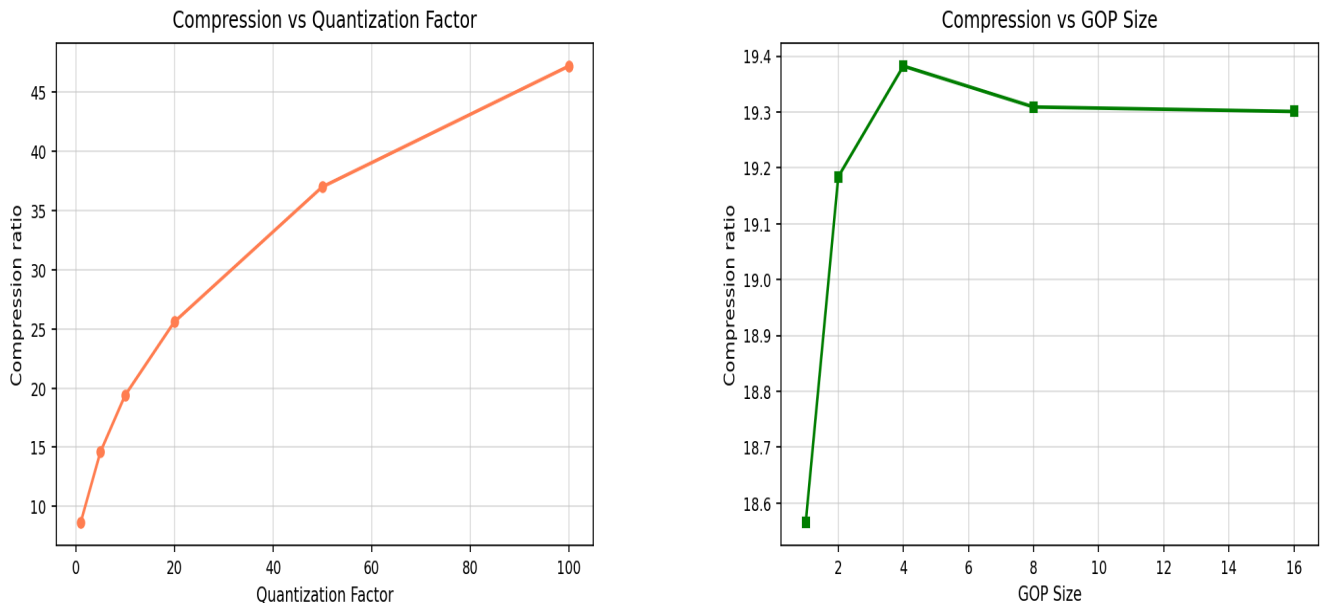


Figure 2: Analyse quantitative : impact du facteur de quantification et du GOP sur la compression MPEG-4

➤ Compression ratio vs Facteur de quantification

Le ratio de compression augmente de façon non linéaire avec le QF : il passe d'environ 9× pour QF=1 à 47× pour QF=100. L'amélioration est surtout marquée entre QF=1 et QF=20, puis elle devient de plus en plus faible au-delà de QF=50, car la majorité des coefficients DCT sont déjà supprimés. Nous avons choisi QF=10 (~19×) comme valeur par défaut, car elle offre un bon équilibre entre compression efficace et qualité visuelle correcte, avec un PSNR de 33,7 dB et peu d'artefacts visibles.

➤ Impact du GOP Size sur la compression

La courbe du GOP est un peu plus complexe. Le ratio commence à 18,6× pour GOP=1 (uniquement des I-frames, sans prédiction temporelle), augmente rapidement jusqu'à un maximum de 19,4× pour GOP=4, puis baisse légèrement à environ 19,3× avant de se stabiliser pour des valeurs plus grandes. Cette petite baisse s'explique par les erreurs qui s'accumulent dans les P-frames sur une séquence courte (17 images) : plus le GOP est grand, plus les P-frames sont loin de leur I-frame de référence, donc les résidus deviennent plus importants et le gain diminue. Ainsi, GOP=4 est le meilleur choix, car il donne une bonne compression tout en limitant les erreurs.

5. Conclusion

Ce projet nous a permis de comprendre concrètement le fonctionnement d'un codec vidéo en implémentant les principales étapes du pipeline MPEG-4.

Chaque étape, de la conversion colorimétrique à la compression entropique, participe à réduire la taille des données tout en conservant une qualité visuelle acceptable.

Les résultats montrent que le facteur de quantification est le paramètre le plus influent sur la compression, tandis que la taille du GOP a un impact plus limité, surtout sur des séquences courtes. Un GOP de 4 avec un QF de 10 donne le meilleur compromis pour notre vidéo de test.

Ce travail nous a aidés à mieux comprendre les enjeux et les choix techniques derrière la compression vidéo moderne.